Login con terceros + roles en .NET MAUI

Guía desde cero para activar los dos caminos: **Google nativo** y **Google asistido por Supabase**

Qué vas a lograr. Una app MAUI (Android y Windows) con dos botones de Google. En ambos, *quién eres* lo confirma un tercero y *que puedes hacer* lo decide una tabla de roles propia (viewer, editor, admin). Esta guía parte de cero: cuentas, consolas, claves, código y pruebas.

	A - Google nativo	B - Asistido por Supabase
Quién autentica	Google, directo (OAuth 2.0 + PKCE)	Supabase, que a su vez usa Google
Servidor tuyo	No hay	Supabase (Auth + Postgres)
Tabla de roles	SQLite local en el dispositivo (local_user_roles)	Postgres en Supabase (public.user_roles) con RLS
Quién protege los roles	El código de la app	El servidor (Row Level Security)
Lo que recibe la app	access_token de Google + perfil	JWT de Supabase (sesión)

Sobre las capturas: son de las consolas reales (Google Cloud y Supabase, en ingles/espanol según cada una) y de la app en el emulador. Se taparon con gris los correos, nombres, avatares y los ultimos caracteres de los secretos. Los textos de la consola pueden variar un poco con el tiempo; guiate por el nombre de cada sección.

Contenido: 1 Requisitos - 2 Proyecto y archivo de configuración - 3 Google Cloud - 4 Camino A - 5 Supabase - 6 Camino B - 7 Roles - 8 Lista de verificación - 9 Errores comunes - 10 Seguridad y Git - Anexo SQL.

1. Requisitos previos

Qué	Para qué	Cómo verificar
Visual Studio 2022/2026 o CLI de .NET 10 con la carga de trabajo MAUI	Compilar la app	dotnet --list-sdks y dotnet workload list (debe aparecer maui / android)
Android SDK + un emulador (AVD) con Google Play o Chrome	Probar en Android	adb devices muestra emulator-5554 en estado device
JDK (incluye keytool)	Obtener el SHA-1 del keystore	keytool -help
Cuenta de Google	Google Cloud y usuario de prueba	Entrar a console.cloud.google.com
Cuenta de Supabase (plan gratuito sirve)	Auth + base de datos de roles	Entrar a supabase.com/dashboard
Cuenta de Google en el emulador o en su Chrome	Iniciar sesión al probar	Chrome del emulador abre accounts.google.com sin error

Límite del plan gratuito de Supabase: solo 2 proyectos activos por cuenta. Si tienes mas, pausa uno (Project Settings > General > Pause project) antes de crear o restaurar el que usaras.

2. Proyecto y archivo de configuración

2.1 Estructura del proyecto

Archivo	Función
Services/GoogleAuthService.cs	Camino A: flujo PKCE directo con Google
Services/SupabaseAuthService.cs	Camino B: login por Supabase (Google y correo/clave) y lectura/edicion de roles
Services/OAuthHelper.cs	Piezas comunes: PKCE, navegador del sistema, recepcion de la redirección
Services/LocalRoleStore.cs	Tabla de roles local (SQLite)
ViewModels/ y Pages/	Pantalla de login y pantalla de inicio (la UI cambia según el rol)
Platforms/Android/WebAuthenticationCallbackActivity.cs	Recibe en Android la vuelta desde Google y desde Supabase
Config/AuthConfig.cs.example	Plantilla de parametros (se sube a Git)
Config/AuthConfig.cs	Tus valores reales (NO se sube; está en .gitignore)
supabase/schema.sql	Tabla de roles, RLS y trigger

2.2 Crear tu AuthConfig.cs

1. Copia **Config/AuthConfig.cs.example** como **Config/AuthConfig.cs** (misma carpeta, sin la extension .example).
2. Reemplaza los valores que empiezan por **TU-** o **TU_** a medida que avances en las secciones 3 y 5.
3. Confirma que **.gitignore** contiene la linea `logins/Config/AuthConfig.cs`.

Parametro	De dónde sale	Sección
SupabaseUrl	Supabase > Project Settings > API > Project URL	5.1
SupabaseAnonKey	Supabase > Project Settings > API > anon / publishable key (NUNCA la service_role)	5.1
SupabaseAndroidRedirectScheme	El ApplicationId de la app: com.companyname.logins	5.4
GoogleClientId / GoogleClientSecret	Cliente OAuth tipo 'App de escritorio' (solo Windows/Mac)	3.5
GoogleAndroidClientId	Cliente OAuth tipo 'Android'	3.4
GoogleAndroidRedirectScheme	com.googleusercontent.apps. + la parte del Client ID Android antes de .apps.googleusercontent.com	3.4
LocalAdminEmails	Correos que serán admin la primera vez que entren por el camino A	7

3. Google Cloud (para los dos caminos)

Consola: **console.cloud.google.com**. Se usa un mismo proyecto de Google Cloud para todos los clientes OAuth.

3.1 Proyecto

1. Arriba a la izquierda abre el selector de proyectos y elige uno existente o pulsa **Proyecto nuevo**.
2. Anota el nombre: lo verás en la pantalla de consentimiento que muestra Google a los usuarios.

3.2 Pantalla de consentimiento (Google Auth Platform)

1. Menú > **APIs y servicios > Pantalla de consentimiento de OAuth** (o 'Google Auth Platform').
2. Si es la primera vez: pulsa **Comenzar**, completa nombre de la app y correo de asistencia, elige **Externo** y acepta.
3. En **Información de la marca** basta con nombre y correos para pruebas.

3.3 Usuarios de prueba (obligatorio mientras la app esté en modo Prueba)

1. Google Auth Platform > **Público**. Verifica que el estado sea **Prueba**.
2. En **Usuarios de prueba** pulsa **Add users**, escribe tu correo exacto de Google y guarda.
3. Revisa que el correo esté bien escrito: una letra de más o de menos hace que Google rechace el acceso.

> Si el usuario no está en esa lista, Google responde 'Acceso bloqueado' o 'access_denied' aunque todo lo demás esté bien.

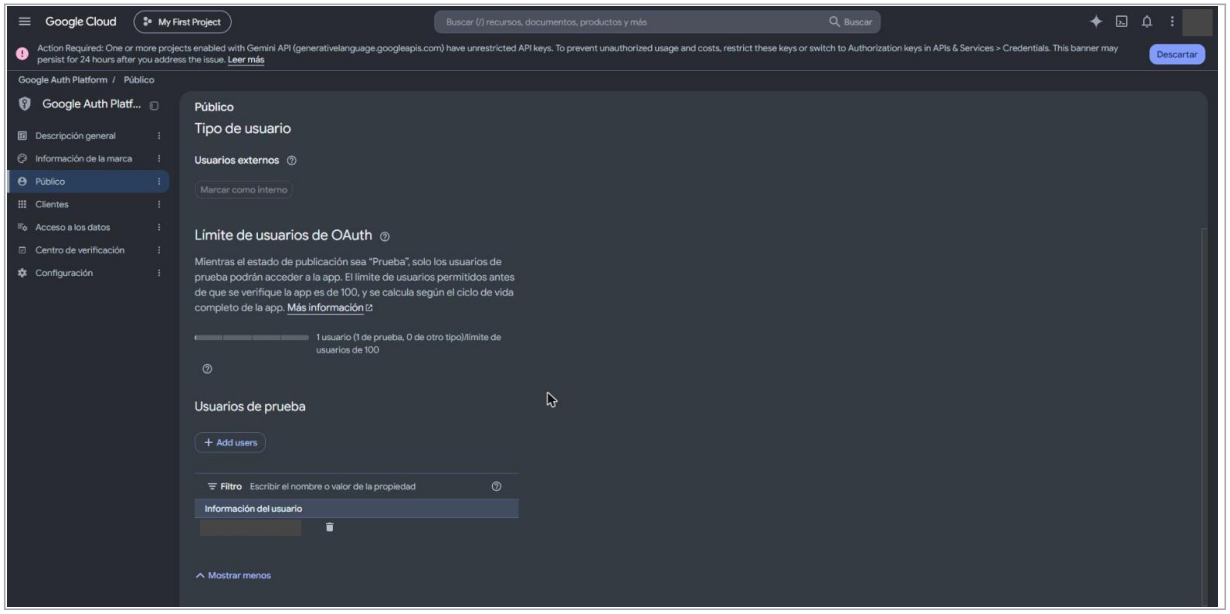


Figura 1. *Google Auth Platform > Público: estado Prueba y lista de usuarios de prueba (el correo esta oculto en la captura).*

3.4 Cliente OAuth tipo Android (camino A en Android)

1. Google Auth Platform > **Clientes > Crear cliente**. Tipo de aplicación: **Android**.
2. **Nombre del paquete:** el ApplicationId de la app (por defecto `com.companyname.logins`; está en el .csproj).
3. **Huella SHA-1:** la del keystore con el que se firma la app. Para depuración en Visual Studio:

```
keytool -list -v -keystore "%LOCALAPPDATA%\Xamarin\Mono for Android\debug.keystore" ^
        -alias androiddebugkey -storepass android -keypass android
```

Copia la linea **SHA1**. Si no tienes keytool en el PATH, está en la carpeta bin del JDK (por ejemplo C:\Program Files\Android\openjdk\jdk-21...\bin).

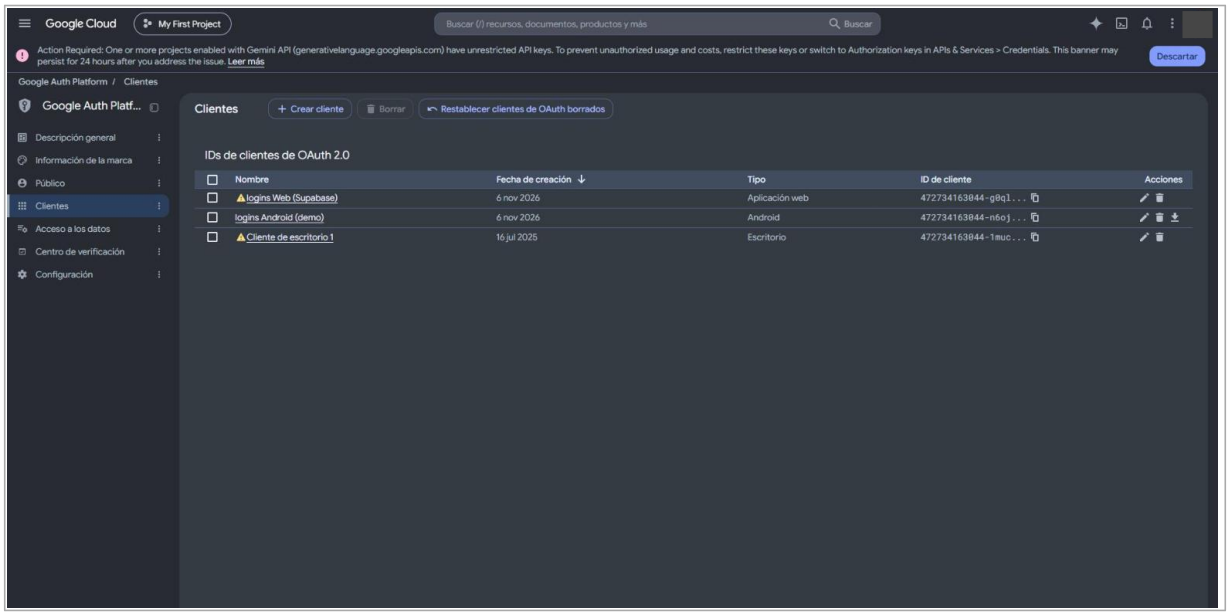


Figura 2. *Clientes > Crear cliente. Aqui verás también los clientes ya creados (Web, Android y Escritorio).*

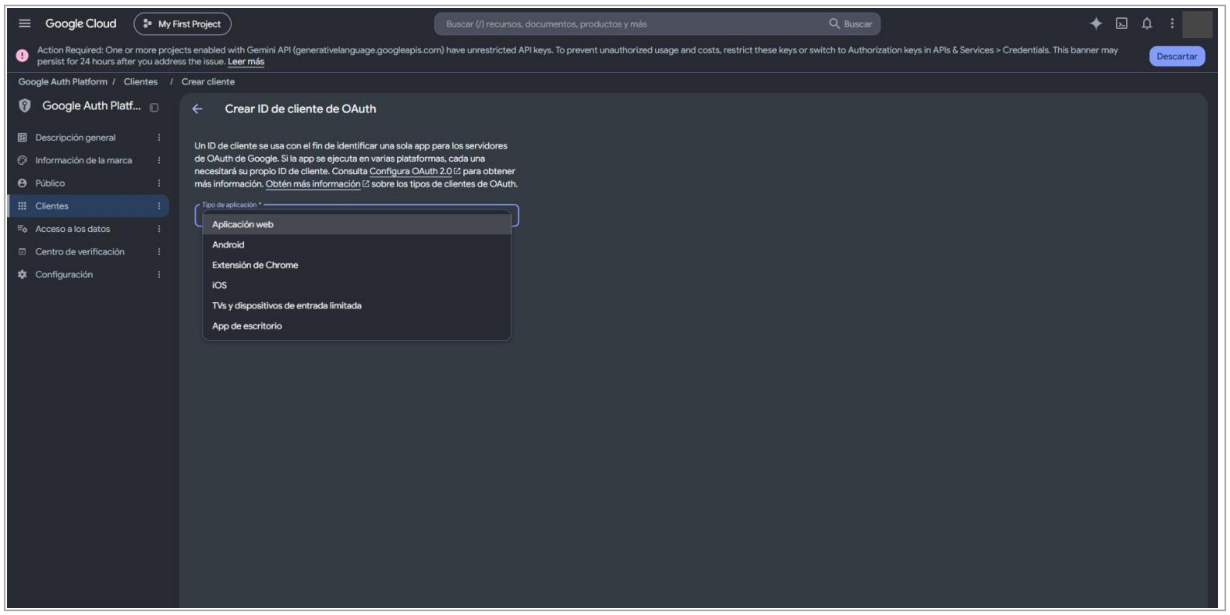


Figura 3. *Al crear el cliente, el menu 'Tipo de aplicación' ofrece Aplicación web, Android y App de escritorio.*

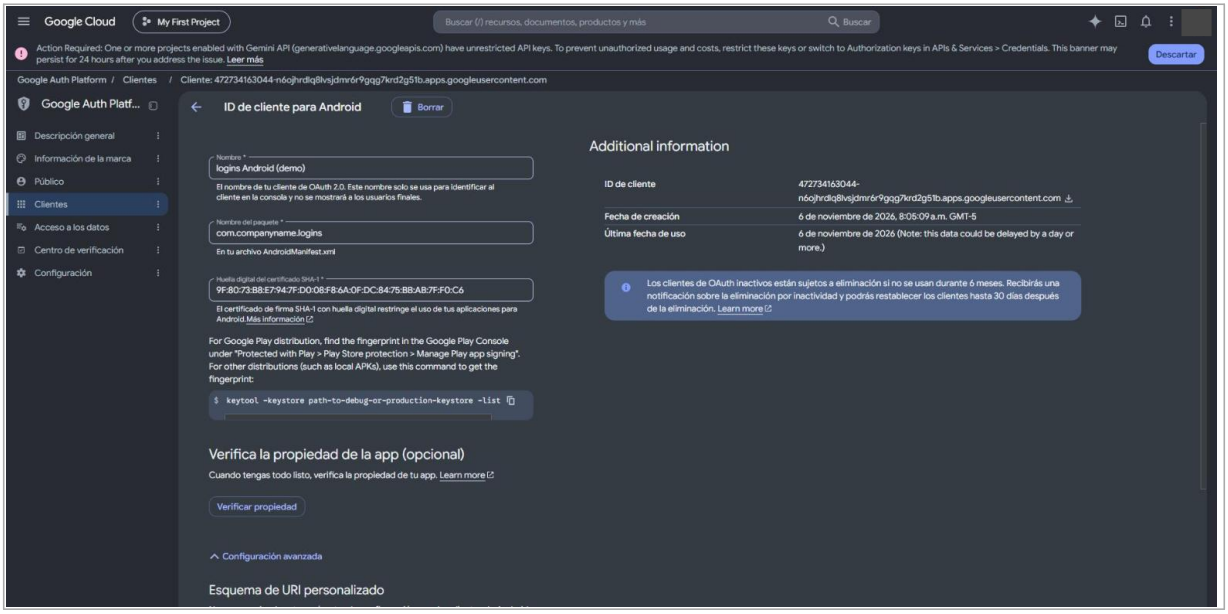


Figura 4. *Cliente Android: nombre, nombre del paquete (ApplicationId) y huella SHA-1 del keystore.*

4. Pulsa **Crear**. Copia el **ID de cliente** (termina en .apps.googleusercontent.com).
5. **Paso que casi todos olvidan:** abre el cliente recién creado > **Configuración avanzada** > marca **Habilitar el esquema de URI personalizado** > **Guardar**. Sin esto Google responde *Custom URI scheme is not enabled for your Android client*.
6. Espera de 5 minutos a unas horas a que el cambio se propague (a veces la consola sigue mostrando la casilla vacía; la prueba real es abrir el login).
7. En AuthConfig.cs: **GoogleAndroidClientId** = ese ID. **GoogleAndroidRedirectScheme** = `com.googleusercontent.apps.` + la parte del ID antes de `.apps.googleusercontent.com`.

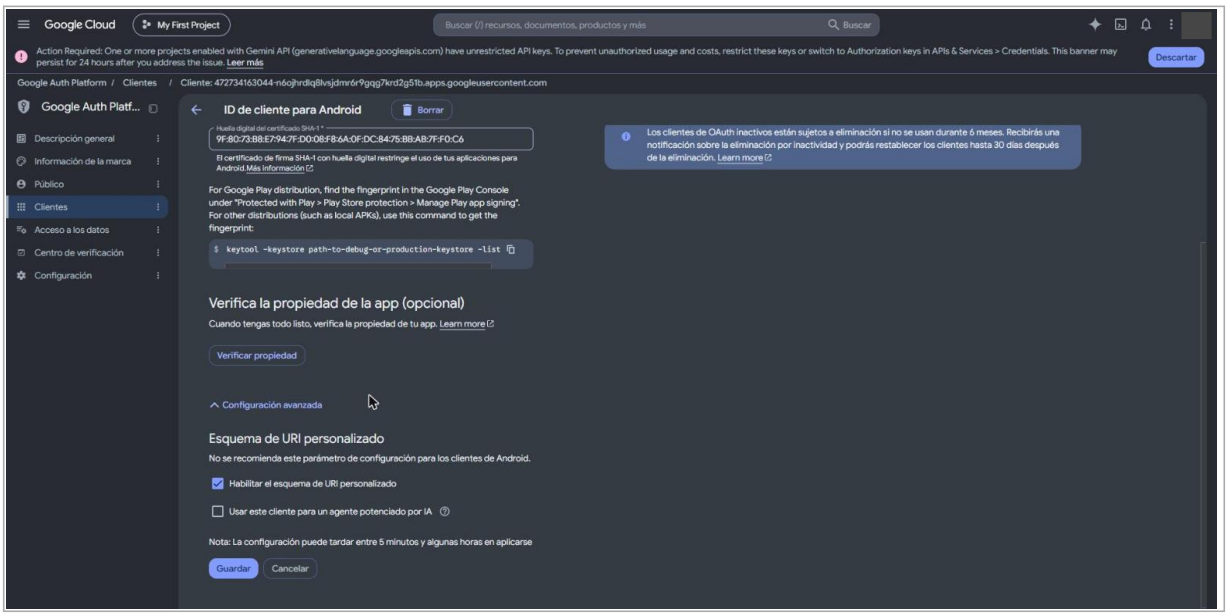


Figura 5. *Configuración avanzada del cliente Android: 'Habilitar el esquema de URI personalizado' debe estar marcado (y guardado).*

> Ejemplo de esquema inverso. Si el ID es `123-abc.apps.googleusercontent.com`, el esquema es `com.googleusercontent.apps.123-abc`.

3.5 Cliente OAuth tipo App de escritorio (camino A en Windows/Mac)

1. Clientes > Crear cliente > Tipo: **App de escritorio**. Dale un nombre y crea.
2. Copia el **ID de cliente** y el **secreto**. Google exige el secreto en este tipo de cliente aunque no sea realmente secreto.
3. En AuthConfig.cs: **GoogleClientId** y **GoogleClientSecret**.
4. No hay que registrar redirecciones: Google permite http://127.0.0.1:{puerto} para este tipo de cliente.

3.6 Cliente OAuth tipo Aplicación web (solo para el camino B)

1. Clientes > Crear cliente > Tipo: **Aplicación web**. Nombre: por ejemplo 'logins Web (Supabase)'.
2. En **URIs de redireccionamiento autorizados** pulsa Agregar URI y escribe EXACTAMENTE: `https://<TU-PROJECT-REF>.supabase.co/auth/v1/callback`
3. Pulsa **Crear**. Se muestra el **ID de cliente** y el **secreto** UNA sola vez: cópialos de inmediato y guárdalos en un gestor de claves.
4. Si cerraste el diálogo sin copiar el secreto: abre el cliente > **Secretos del cliente > + Add secret** y copia el nuevo (el anterior solo muestra ****xxxx).

> Este cliente Web y su secreto se pegan en **Supabase** (sección 5.5). Nunca van dentro de la app ni en Git.

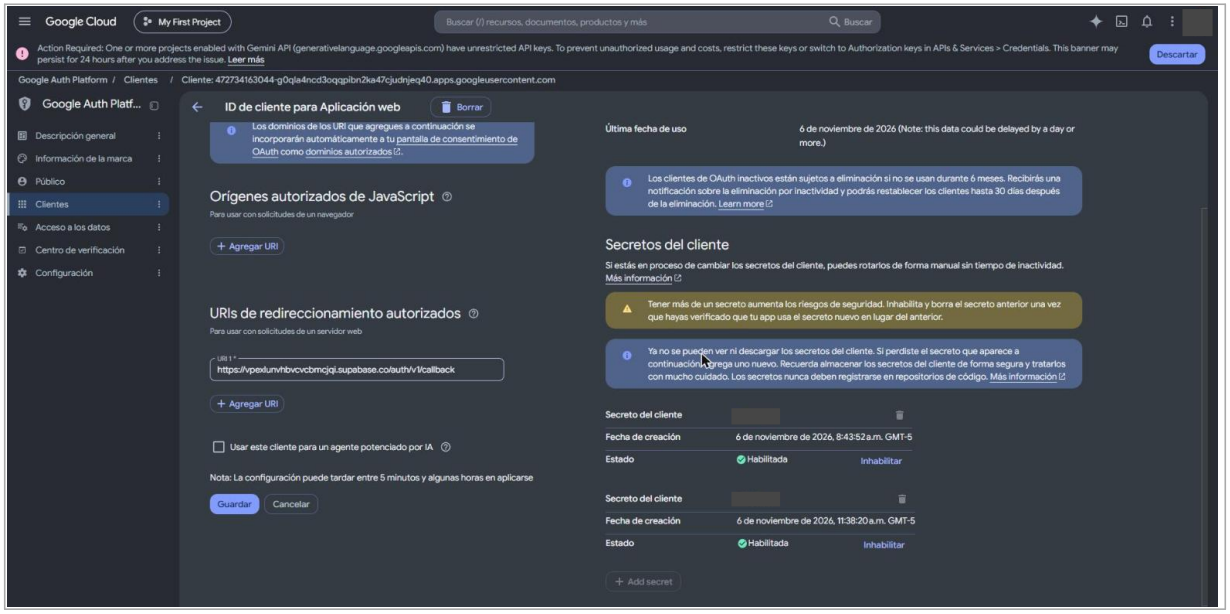


Figura 6. *Cliente Web: el URI de redireccionamiento autorizado es el callback de Supabase; a la derecha, la sección de secretos (solo muestra los ultimos caracteres, ocultos aqui).*

4. Camino A - Google nativo

4.1 Cómo funciona

1. La app genera un **code_verifier** aleatorio y su **code_challenge** (SHA-256).
2. Abre el navegador del sistema en accounts.google.com con el challenge. La app nunca ve la clave de Google.
3. Google redirige a la app con un **code** (Android: esquema com.googleusercontent.apps...; Windows: http://127.0.0.1:{puerto}).
4. La app cambia code + code_verifier por tokens en oauth2.googleapis.com/token y pide el perfil a /userinfo.
5. El rol se busca (o se crea la primera vez) en la tabla SQLite local: admin si el correo está en LocalAdminEmails; si no, viewer.

4.2 Activarlo - lista corta

1. Completa 3.1 a 3.4 (Android) y/o 3.5 (Windows).
2. Rellena **GoogleAndroidClientId**, **GoogleAndroidRedirectScheme** (y los de escritorio si aplica) en AuthConfig.cs.
3. Escribe tu correo en **LocalAdminEmails** si quieres ser admin.
4. Compila y despliega: `dotnet build -f net10.0-android -t:Run`
5. En el emulador toca **Continuar con Google** en la tarjeta A. Inicia sesión (la primera vez en el Chrome del emulador) y acepta los permisos.
6. La app vuelve sola y muestra 'Google nativo', tu nombre y tu rol.

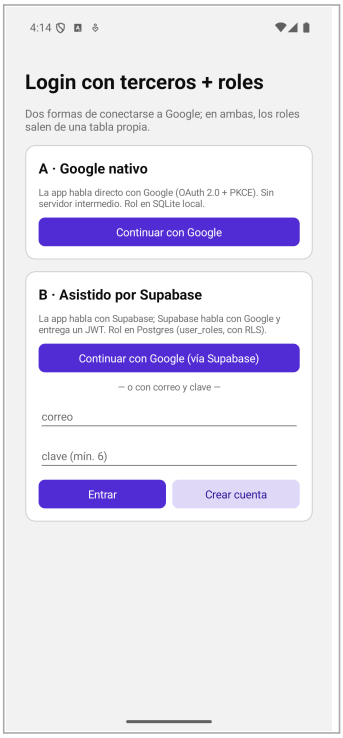


Figura 7. Pantalla de login: tarjeta A (Google nativo) y tarjeta B (Supabase).

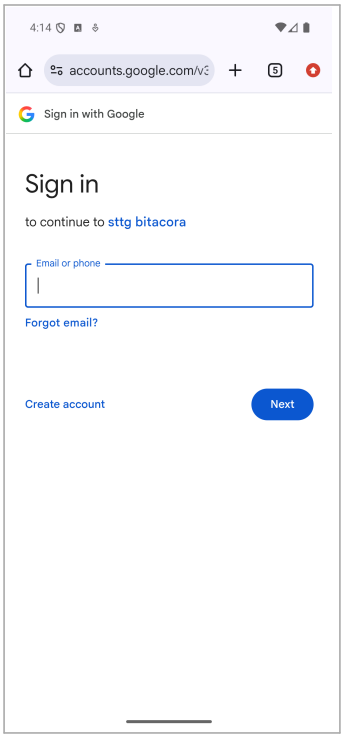


Figura 8. Chrome del emulador pide iniciar sesión en Google (la primera vez).

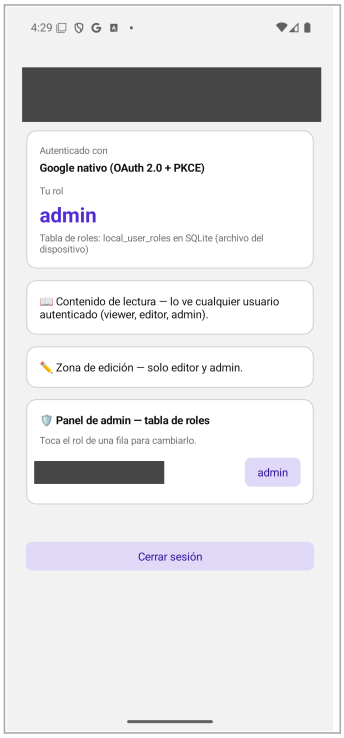


Figura 9. Resultado del camino A: 'Google nativo', rol admin desde SQLite local (datos personales ocultos).

4.3 Si algo falla

Síntoma	Causa probable	Solución
invalid_request - Custom URI scheme is not enabled	Falta habilitar el esquema de URI personalizado	3.4 paso 5; esperar propagación
Acceso bloqueado / access_denied	Correo no está en usuarios de prueba	3.3
Google no devuelve a la app	Esquema inverso mal escrito en AuthConfig	Compararlo con el Client ID Android (3.4)
'Google no está configurado'	AuthConfig aún con valores TODO/TU_	Completar la configuración (2.2)

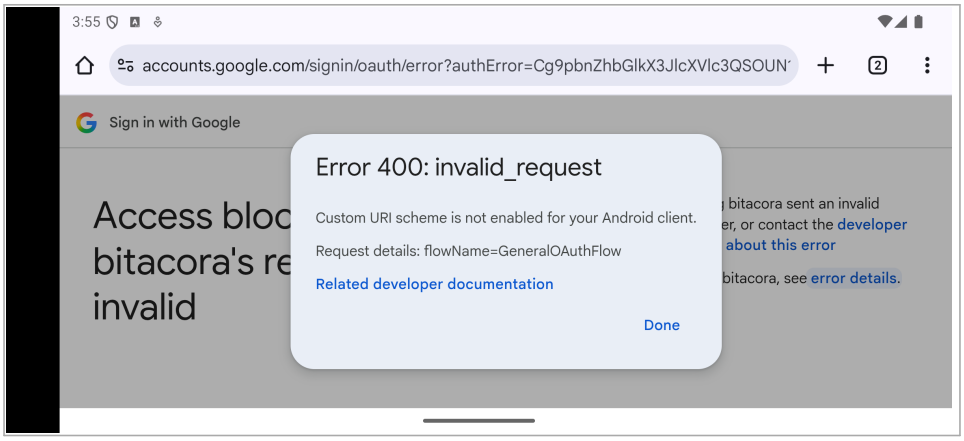


Figura 10. *Error tipico si falta habilitar el esquema de URI personalizado (detalle del error 400 invalid_request).*

5. Supabase (para el camino B)

5.1 Proyecto, URL y clave

1. En supabase.com/dashboard crea un proyecto (o restaura uno pausado). Espera a que el estado sea **Active**.
2. **Project Settings > API:** copia **Project URL** y la clave **anon / publishable**.
3. Pégalos en AuthConfig.cs (**SupabaseUrl**, **SupabaseAnonKey**). La anon key es pública por diseño; la seguridad la da RLS. La **service_role** jamas va en una app.

5.2 Crear la tabla de roles (SQL Editor)

1. Abre **SQL Editor > New query**.
2. Pega el contenido de **supabase/schema.sql** (está completo en el Anexo de esta guía) y pulsa **Run**.
3. Verifica en **Table Editor** que existe **public.user_roles** con RLS activado.

Qué hace el script: crea user_roles (una fila por usuario), activa RLS, define 3 políticas (cada uno lee su fila; el admin lee todas; el admin edita roles), una función is_admin() y un trigger que asigna 'viewer' a cada usuario nuevo de auth.users.

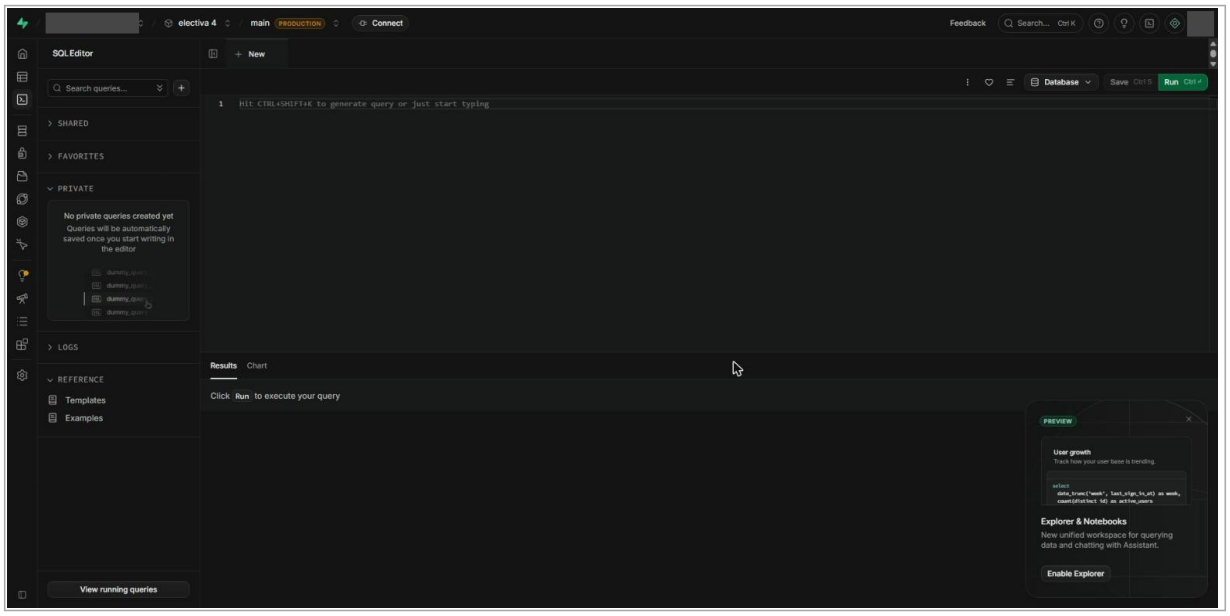


Figura 11. *Supabase > SQL Editor > New query: pega aqui el contenido de schema.sql y pulsa Run.*

5.3 Opciones de autenticación por correo (opcional)

1. **Authentication > Sign In / Providers > Email:** deja activado el proveedor.
2. Para pruebas rápidas con 'Crear cuenta' puedes desactivar **Confirm email**; si lo dejas activo, el usuario debe confirmar desde su correo antes de entrar.

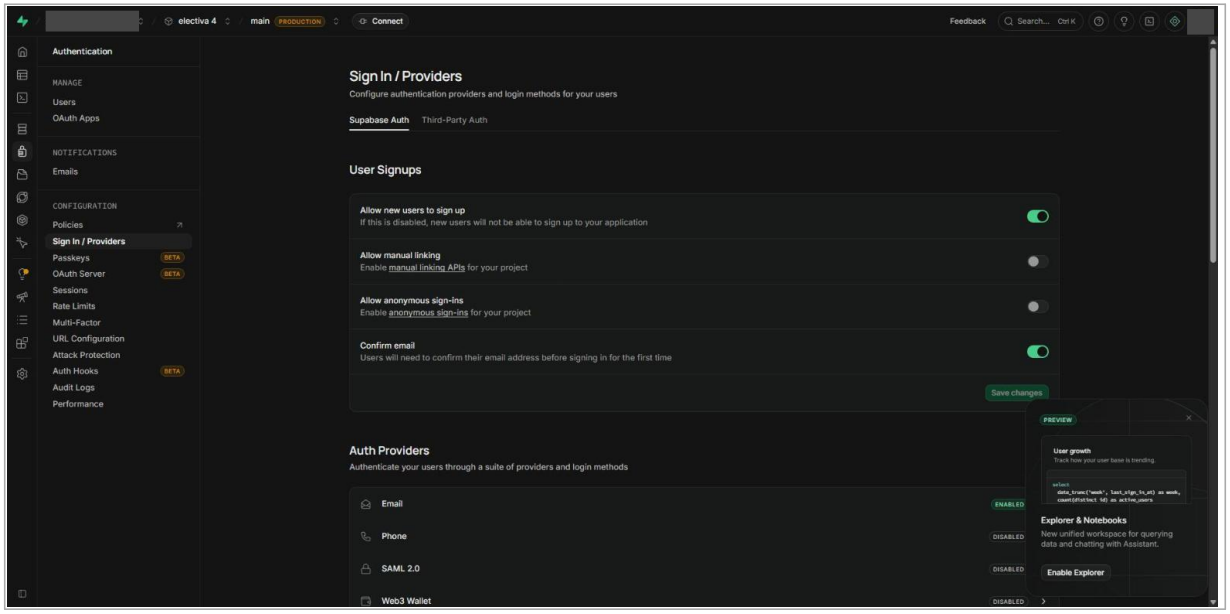


Figura 12. *Authentication > Sign In / Providers: interruptor 'Confirm email' y lista de proveedores (Email activo).*

5.4 URL de redirección permitidas (obligatorio para Google via Supabase)

1. **Authentication > URL Configuration > Redirect URLs > Add URL.**
2. Agrega: `com.companyname.logins://callback` (Android; usa tu ApplicationId).
3. Agrega: `http://127.0.0.1:**` (Windows/Mac; el comodin ** admite cualquier puerto).
4. Pulsa **Save URLs** y comprueba que aparecen en la lista.

> Si una URL no está en la lista, Supabase ignora el redirect_to y manda al usuario al Site URL, y la app no recibe el código.

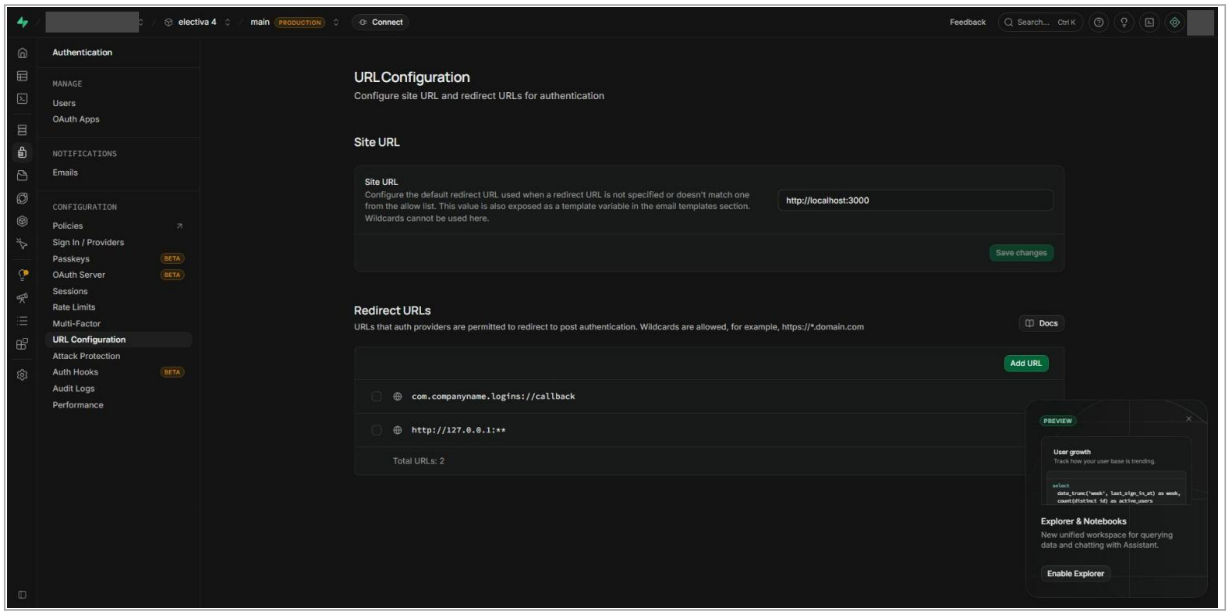


Figura 13. *Authentication > URL Configuration: las dos Redirect URLs permitidas (esquema de la app y 127.0.0.1 con comodin).*

5.5 Activar el proveedor Google

1. **Authentication > Sign In / Providers > Google.**
2. Activa **Enable Sign in with Google**.

3. En **Client IDs** pega el ID del cliente **Web** (3.6).
4. En **Client Secret (for OAuth)** pega el secreto del cliente Web. Pulsa **Reveal** para confirmar que empieza por GOCSPX-.
5. Comprueba que la **Callback URL** mostrada es la misma que registraste en Google (3.6).
6. Pulsa **Save** y espera el aviso 'Successfully updated settings'.

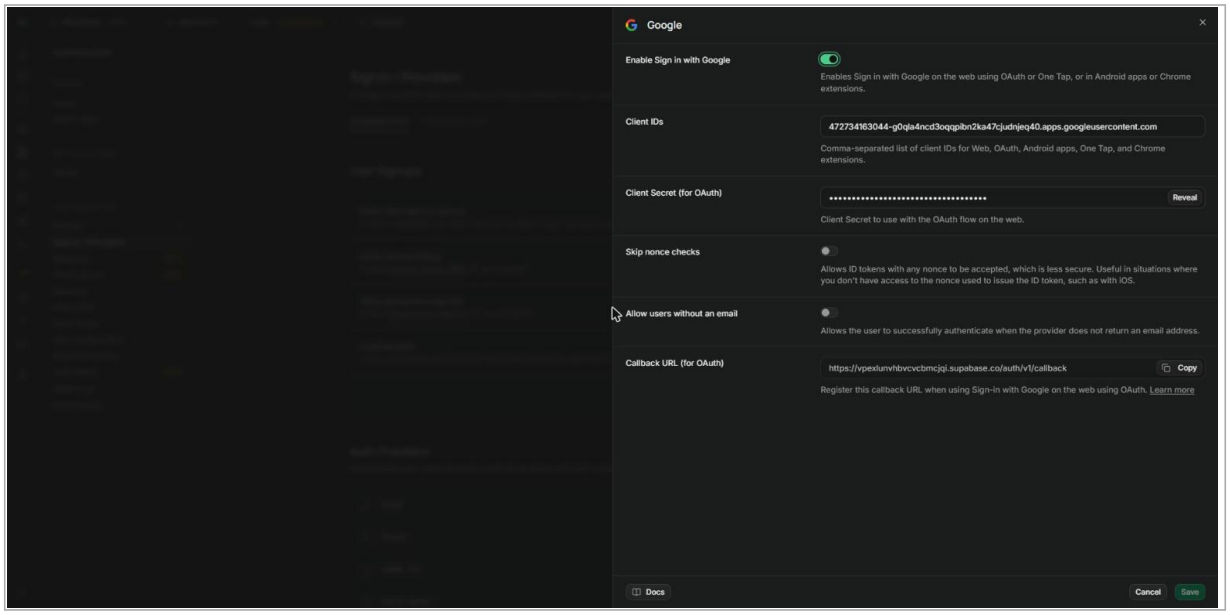


Figura 14. *Panel del proveedor Google en Supabase: interruptor activo, Client ID del cliente Web, Client Secret (enmascarado) y Callback URL.*

Comprobar que quedo bien (sin navegador)

```
curl.exe -s -D - "https://<TU-PROJECT-REF>.supabase.co/auth/v1/authorize?provider=google&redirect_to=http%3A%2F%2F1
```

- **HTTP 302** con Location que apunta a accounts.google.com: correcto.
- **400 'provider is not enabled'**: el interruptor está apagado o no guardaste.
- **400 'missing OAuth secret'**: el campo Client Secret está vacío o no se guardó.

6. Camino B - Asistido por Supabase

6.1 Cómo funciona

1. La app genera PKCE y abre {SupabaseUrl}/auth/v1/authorize?provider=google&redirect_to=...&code_challenge=...
2. Supabase redirige a Google usando SU cliente Web (el de 3.6). El usuario inicia sesión en Google.
3. Google devuelve el control a Supabase (/auth/v1/callback). Supabase crea o actualiza el usuario en auth.users (el trigger crea su fila en user_roles).
4. Supabase redirige a la app con ?code=...
5. La app canjea el code por la sesión en /auth/v1/token?grant_type=pkce y recibe el JWT.
6. Con ese JWT lee public.user_roles; Postgres aplica RLS con auth.uid() y devuelve solo lo permitido.

6.2 Activarlo - lista corta

1. Completa 3.1 a 3.3 y 3.6 (Google) y 5.1, 5.2, 5.4, 5.5 (Supabase).
2. Rellena SupabaseUrl y SupabaseAnonKey en AuthConfig.cs.
3. Compila y despliega en el emulador.
4. Toca **Continuar con Google (vía Supabase)**. Elige tu cuenta.

5. La app vuelve y muestra 'Asistido por Supabase'. El primer ingreso es **viewer**; para ser admin ve a la sección 7.

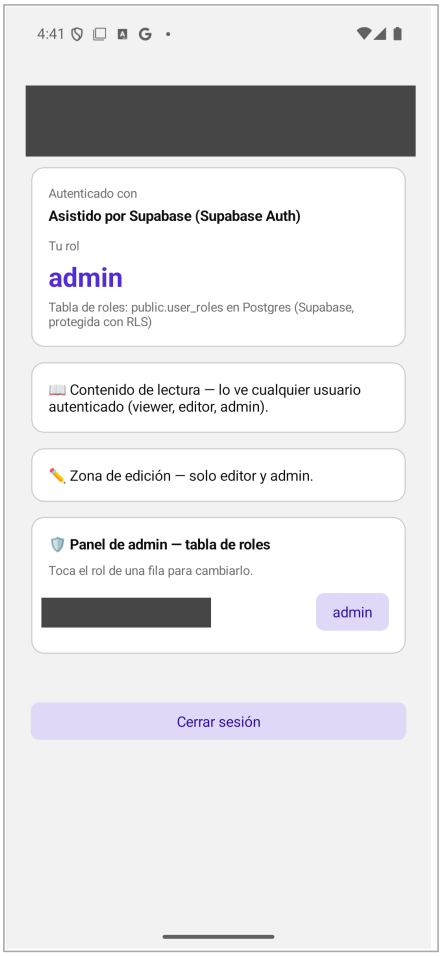


Figura 15. *Resultado del camino B con rol admin: 'Asistido por Supabase', tabla public.user_roles y panel de administracion (datos personales ocultos).*

6.3 Si algo falla

Síntoma	Causa probable	Solución
400 provider is not enabled	Proveedor Google apagado o sin guardar	5.5
400 missing OAuth secret	Client Secret vacío	3.6 (nuevo secreto) y 5.5
redirect_uri_mismatch (en Google)	El URI del cliente Web no coincide exacto con la Callback URL de Supabase	3.6 paso 2
La app no recibe el código / abre otra página	Redirect URL no está en la lista permitida	5.4
'No se pudo cambiar el rol'	Tu usuario no es admin según user_roles	Sección 7
No puedo crear/restaurar el proyecto	Límite de 2 proyectos activos del plan gratuito	Pausar otro (Requisitos)

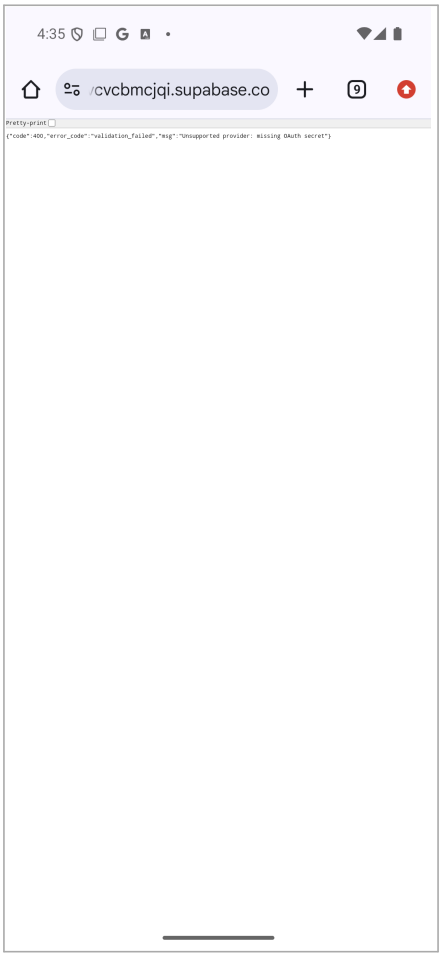


Figura 16. *Respuesta de Supabase cuando el proveedor Google no tiene Client Secret guardado.*

7. Roles: como se asignan y cómo cambian

Rol	Qué ve en la app
viewer	Contenido de lectura
editor	Lectura + zona de edicion
admin	Lectura + edicion + panel que lista y cambia los roles de todos

7.1 Camino A (SQLite local)

- Primer ingreso: la fila se crea sola. Si el correo está en **LocalAdminEmails** es admin; si no, viewer.
- Para cambiar roles: entra como admin y toca el rol de la fila en el panel de admin.
- Si cambias LocalAdminEmails después del primer ingreso, no surte efecto (la fila ya existe). Borra los datos de la app o edita la fila.
- La proteccion es de código: solo la app impide que un viewer cambie roles. Sirve para aprender; en producción los roles deben vivir en un servidor.

7.2 Camino B (Postgres con RLS)

- Primer ingreso: el trigger crea la fila con rol **viewer**.
- Convertir al primer admin (una sola vez, en el SQL Editor de Supabase):

```
update public.user_roles set role = 'admin' where email = 'tu-correo@gmail.com';
```

- Después, un admin cambia roles desde el panel de la app; la política 'admin cambia roles' lo permite y RLS bloquea a cualquier otro.
- El cambio se ve al volver a iniciar sesión (el rol se lee en el login).

8. Lista de verificación final

	Comprobación	Camino
[ ]	Usuario de prueba agregado en Google Auth Platform > Público	A y B
[ ]	Cliente Android con paquete + SHA-1 y esquema de URI personalizado habilitado	A (Android)
[ ]	AuthConfig.cs con GoogleAndroidClientId y esquema inverso correctos	A (Android)
[ ]	Cliente Web con URI de redirección = callback de Supabase	B
[ ]	schema.sql ejecutado; user_roles existe con RLS	B
[ ]	Redirect URLs agregadas en Supabase (esquema de la app y 127.0.0.1)	B
[ ]	Proveedor Google activo con Client ID y Client Secret guardados (curl da 302)	B
[ ]	AuthConfig.cs con SupabaseUrl y SupabaseAnonKey	B
[ ]	Prueba A: entra, muestra 'Google nativo' y el rol local	A
[ ]	Prueba B: entra, muestra 'Asistido por Supabase' y el rol de Postgres	B
[ ]	Cambiar un usuario a admin y comprobar que aparece el panel de admin	A y B

9. Errores comunes (los que realmente aparecen)

Mensaje	Significa	Arreglo
Error 400: invalid_request - Custom URI scheme is not enabled for your Android client	Cliente Android sin el esquema de URI personalizado	3.4 paso 5
Unsupported provider: provider is not enabled	Google apagado en Supabase	5.5
Unsupported provider: missing OAuth secret	Falta el Client Secret en Supabase	3.6 y 5.5
Access blocked / access_denied	Correo fuera de usuarios de prueba	3.3
redirect_uri_mismatch	URI de redirección distinto en el cliente Web	3.6
Cannot restore project: limit of active free projects	Máximo 2 proyectos gratis activos	Pausar otro proyecto
Cannot pause project while it is hibernating	El proyecto está hibernando; no se puede pausar desde la API	Hacerlo desde el dashboard o soporte
El botón no responde y aparece 'X no está configurado'	AuthConfig sin completar	2.2

10. Seguridad y Git

- **Se sube a Git:** AuthConfig.cs.example, schema.sql, el código y esta guía.
- **NO se sube:** AuthConfig.cs (ya en .gitignore), secretos de clientes, la service_role de Supabase, archivos JSON descargados de Google.
- Los secretos de Google solo viven en Google Cloud y en el panel de Supabase; nunca en la app ni en el chat.
- El client secret del cliente de escritorio no es realmente secreto (está en el binario), pero igual no se pública.
- Rota (Add secret / eliminar el anterior) cualquier secreto que se haya expuesto.
- Antes de publicar la app, pasa la pantalla de consentimiento a Producción o mantente en Prueba con usuarios de prueba (límite 100).
- Para producción: firma la app con tu keystore de release, crea otro cliente Android con ese SHA-1 y no uses el esquema de URI personalizado si Google ofrece alternativa (Credential Manager).

Anexo - schema.sql completo

```
create table if not exists public.user_roles (
  user_id    uuid primary key references auth.users(id) on delete cascade,
  email      text not null,
  role       text not null default 'viewer' check (role in ('admin','editor','viewer')),
  created_at timestamptz not null default now()
);

alter table public.user_roles enable row level security;

create or replace function public.is_admin()
returns boolean language sql security definer set search_path = public stable
as $$
  select exists (select 1 from public.user_roles where user_id = auth.uid() and role = 'admin');
$$;

create policy "cada usuario lee su propio rol"
  on public.user_roles for select using (user_id = auth.uid());

create policy "admin lee todos los roles"
  on public.user_roles for select using (public.is_admin());

create policy "admin cambia roles"
  on public.user_roles for update
  using (public.is_admin()) with check (public.is_admin());

create or replace function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = public
as $$
begin
  insert into public.user_roles (user_id, email, role) values (new.id, new.email, 'viewer');
  return new;
end;
$$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

Comandos útiles

```
dotnet build -f net10.0-android -t:Run             # compilar, instalar y lanzar en el emulador/dispositivo
dotnet build -f net10.0-windows10.0.19041.0        # compilar para Windows
adb devices                                         # ver emuladores/dispositivos
adb shell monkey -p com.companyname.logins -c android.intent.category.LAUNCHER 1   # abrir la app
adb shell am force-stop com.companyname.logins      # cerrar la app
```